INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DO RIO GRANDE DO NORTE
Desenvolvimento de Sistemas Distribuídos
Prof. Gracon Lima

TRANSMISSÃO DE DADOS COM OS PROTOCOLOS TCP E UDP

Alice Lima
alice.lima1@escolar.ifrn.edu.br

PROTOCOLOS

TCP e UDP são protocolos da camada de transporte e são usados de acordo com os requisitos do sistema.

TCP

Maior Confiabilidade

Este protocolo, uma vez estabelecida a conexão, permite a transmissão bidirecional de dados. Possui sistemas integrados para verificação de erros e garante a entrega dos dados na ordem enviada, tornando-o ideal para transferir informações como imagens estáticas, arquivos de dados e páginas da web.

UDP

Mais Velocidade

Este é um protocolo de Internet simples e sem conexão, que não requer verificação ou recuperação de erros. Não há sobrecarga para abrir, manter ou encerrar uma conexão, e os dados são enviados continuamente, independentemente de serem recebidos ou não.

SOBRE O PROJETO

Para a interface gráfica foi utilizada a biblioteca tkinter e o ttk para estilização

Aplicação desenvolvida em Python, para transmissão de arquivos, utilizando os protocolos TCP e UDP.

CÓDIGO

01. Biblioteca para criação da interface gráfica
02. Módulos para diálogos de arquivos e mensagens
03. Módulo para estilização e modernização
04. Biblioteca para criar e gerenciar threads
05. Biblioteca para operações de rede
06. Biblioteca para interações com o sistema operacional

07. Tamanho máximo do pacote UDP

08. Flag para controlar o estado do servidor TCP.
09. Flag para controlar o estado do servidor UDP.

```python
import tkinter as tk
from tkinter import filedialog, messagebox
from tkinter import ttk
import threading
import socket
import os

# Tamanho máximo do pacote UDP
UDP_PACKET_SIZE = 1024

# Variáveis globais para controle do servidor
server_tcp_running = False
server_udp_running = False
```

CÓDIGO

- Cria um socket TCP para comunicação;
- Associa o socket a um endereço e porta, e configura o servidor para ouvir conexões;
- Aceita conexões de clientes em um loop infinito;
- Recebe dados do cliente em blocos de 1024 bytes;
- Processa os dados recebidos para extrair o nome e o conteúdo do arquivo e salva o arquivo;
- Fecha a conexão com o cliente após a recepção completa dos dados;
- Captura e exibe erros que possam ocorrer durante a execução.
- Encerra a execução do servidor.

```python
def start_tcp_server():
    global server_tcp_running
    if not server_tcp_running:
        try:
            server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            server.bind(("0.0.0.0", 9999))
            server.listen(5)
            print("Servidor TCP esperando por conexões...")

            while True:
                client_socket, addr = server.accept()
                print(f"Conexão de {addr} estabelecida.")
                file_data = b""
                while True:
                    data = client_socket.recv(1024)
                    if not data:
                        break
                    file_data += data
                file_name, file_content = process_received_data(file_data)
                save_file(file_name, file_content)
                client_socket.close()
        except Exception as e:
            print(f"Erro no Servidor TCP: {e}")
        finally:
            server_tcp_running = False
```

protocolo IPV4

protocolo TCP

CÓDIGO

- Cria e configura um socket UDP.
- Recebe pacotes de dados enviados por clientes.
- Reconstrói os dados na ordem correta.
- Salva o arquivo recebido no sistema.
- Lida com erros e mantém o controle do estado do servidor.

```python
def start_udp_server():
    global server_udp_running
    if not server_udp_running:
        try:
            server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
            server.bind(("0.0.0.0", 9999))
            print("Servidor UDP esperando por dados...")

            file_data = {}
            expected_packets = 0
            received_packets = 0
            file_name = None
```

```python
while True:
    data, addr = server.recvfrom(UDP_PACKET_SIZE + 100)
    if data:
        try:
            parts = data.split(b'|')
            if len(parts) < 4:
                raise ValueError("Cabeçalho incompleto")

            if file_name is None:
                file_name = parts[0].decode()
                total_packets = int(parts[1].decode())
                expected_packets = total_packets

            seq_num = int(parts[2].decode())
            file_data[seq_num] = parts[3]

            received_packets = len(file_data)
            if received_packets == expected_packets:
                ordered_data = b''.join(file_data[i] for i in range(expected_packets))
                save_file(file_name, ordered_data)
                print(f"Arquivo '{file_name}' recebido e salvo com sucesso pelo UDP.")
                break

        except ValueError as ve:
            print(f"Erro ao processar cabeçalho UDP: {ve}")
            continue
```

```python
except Exception as e:
    print(f"Erro no Servidor UDP: {e}")
finally:
    server_udp_running = False
```

CÓDIGO

- Extração do nome e conteúdo do arquivo dos dados recebidos, separando-os para serem utilizados posteriormente.

- Os dados são divididos, usando | como delimitador, em sua Iª ocorrência. A primeira parte é decodificada de bytes para string, com o nome do arquivo.

```python
def process_received_data(data):
    file_name, file_content = data.split(b'|', 1)
    return file_name.decode(), file_content
```

- A função retorna o nome do arquivo e o conteúdo restante dos dados binários.

```python
def save_file(file_name, file_data):
    try:
        save_dir = "received_files"
        if not os.path.exists(save_dir):
            os.makedirs(save_dir)
        file_path = os.path.join(save_dir, file_name)
        with open(file_path, "wb") as f:
            f.write(file_data)
        print(f"Arquivo '{file_name}' recebido e salvo com sucesso!")
    except Exception as e:
        print(f"Erro ao salvar o arquivo: {e}")
```

- Salva o arquivo recebido no diretório received_files (caso ainda não exista, ele criará)

CÓDIGO

- Conecta ao servidor TCP, lê o arquivo, envia o nome e o conteúdo do arquivo para o servidor

- Criação do socket usando o protocolo IPv4;
- Estabelece conexão com o servidor TCP;

- Abre o arquivo no modo leitura binária;
- Lê todo conteúdo do arquivo binário;
- Obtém o nome do arquivo e codifica em bytes;

- Envia os dados concatenados para o servidor TCP;
- Garante que todos os dados sejam enviados de uma vez, esperando até que todos tenham sido enviados.
- Fecha o socket cliente após a transmissão.

```python
def start_tcp_client(file_path):
    try:
        client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        client.connect(("127.0.0.1", 9999))
        with open(file_path, "rb") as f:
            file_data = f.read()
            file_name = os.path.basename(file_path).encode()
            client.sendall(file_name + b'|' + file_data)
        client.close()
        print("Arquivo enviado com sucesso pelo TCP!")
    except Exception as e:
        print(f"Erro no Cliente TCP: {e}")
```

CÓDIGO

- Divide o arquivo em pacotes, adiciona cabeçalhos e envia cada pacote para o servidor UDP.

- Criação do socket usando o protocolo IPv4;

- Abre o arquivo no modo leitura binária;
- Lê todo conteúdo do arquivo binário;
- Obtém o nome do arquivo e codifica em bytes;
- Calcula quantos pacotes são necessários para transmitir todos os dados do arquivo (tamanho total do arquivo/tamanho máximo do pacote UDP)

- Criação de loop para iterar sobre cada pacote necessário;
- Definição de posição inicial e final do bloco de dados atual do arquivo;
- Extração do bloco de dados atual do arquivo;
- Criação do cabeçalho para cada pacote (nome do arquivo, o número total de pacotes e o número do pacote atual, codificados em bytes)

```python
def start_udp_client(file_path):
    try:
        client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        with open(file_path, "rb") as f:
            file_data = f.read()
            file_name = os.path.basename(file_path).encode()
            total_packets = (len(file_data) + UDP_PACKET_SIZE - 1) // UDP_PACKET_SIZE

            for i in range(total_packets):
                start = i * UDP_PACKET_SIZE
                end = start + UDP_PACKET_SIZE
                packet = file_data[start:end]
                header = f"{file_name.decode()}|{total_packets}|{i}|".encode()
                client.sendto(header + packet, ("127.0.0.1", 9999))

        print("Arquivo enviado com sucesso pelo UDP!")
    except Exception as e:
        print(f"Erro no Cliente UDP: {e}")
```

- Envia o pacote UDP contendo o cabeçalho concatenado com o bloco de dados para o servidor UDP.

CÓDIGO

- **Permite ao usuário selecionar um arquivo através de uma caixa de diálogo, exibe o caminho completo desse arquivo em um campo de entrada de texto na interface gráfica.**

```python
def select_file():
    file_path = filedialog.askopenfilename()
    file_entry.delete(0, tk.END)
    file_entry.insert(0, file_path)
```

```python
def start_transmission():
    protocol = protocol_var.get()
    file_path = file_entry.get()

    if not file_path:
        messagebox.showwarning("Aviso", "Selecione um arquivo para enviar.")
        return

    if protocol == "TCP":
        threading.Thread(target=start_tcp_client, args=(file_path,)).start()
    elif protocol == "UDP":
        threading.Thread(target=start_udp_client, args=(file_path,)).start()
```

- **Coordena a interface entre a escolha do protocolo de transmissão pelo usuário e a execução das funções específicas em threads separadas.**

REFERÊNCIAS

- Qual a diferença entre TCP e UDP? Entenda o que são esses protocolos!. Disponível em: <https://gaea.com.br/diferenca-entre-tcp-e-udp/>. Acesso em: 2024